Dijkstra's Algorithm

Primary Solutions for Single-Source Shortest Paths

1. Introduction to Dijkstra's Algorithm

Invented in 1956 by Edsger W. Dijkstra, this is the most famous algorithm for solving the **Single-Source Shortest Path (SSSP)** problem. It finds the shortest distance from a starting vertex to every other vertex in a weighted graph.

> **Universal Constraint:** Dijkstra's algorithm only works on graphs with **non-negative edge weights**. For graphs with negative weights, the Bellman-Ford algorithm must be used.

2. How it Works: The Greedy Strategy

The algorithm follows these logical steps:

1. **Initialization:** Set distance to source = 0, and all other distances to Infinity.
2. **Selection:** Pick the unvisited vertex with the smallest known distance.
3. **Relaxation:** For all neighbors of the current vertex, calculate potential new distances. If the new path is shorter, update the distance.
4. **Finalization:** Mark the current vertex as 'visited' and repeat until all nodes are processed.

3. Python Implementation (Core Logic)

```
def dijkstra(self, start_node):
    distances = [float('inf')] * self.size
    distances[start_node] = 0
    visited = [False] * self.size

    for _ in range(self.size):
        # 1. Find the nearest unvisited node
        u = self.min_distance_node(distances, visited)
        visited[u] = True

        # 2. Relax edges
        for v in range(self.size):
            if self.adj_matrix[u][v] > 0 and not visited[v]:
                alt = distances[u] + self.adj_matrix[u][v]
                if alt < distances[v]:
                    distances[v] = alt
```

4. Path Reconstruction

While basic Dijkstra finds the *cost* (e.g., Distance: 10), we often need the *path* (e.g., D -> E -> C -> B -> F). This is achieved using a **Predecessor Array**. Every time an edge is relaxed, we store the 'parent' node that provided the shorter path.

> **Backtracking:** To find the path to F, look at `predecessor[F]`, then `predecessor[that_node]`, until you reach the source.

5. Time Complexity Analysis

The efficiency of Dijkstra's depends on the data structure used to find the minimum distance node:

- **Adjacency Matrix + Array:** $O(V^2)$ — Best for dense graphs.
- **Adjacency List + Min-Heap:** O(E log V) — Standard for sparse graphs.
- **Fibonacci Heap:** O(E + V log V) — Theoretically optimal for very large graphs.

Manual Execution & Trace Table (Part 1)

Professional implementation requires tracking state across multiple iterations. Consider a graph where D is the source. The trace table maintains the **Priority Queue** (or distance array state) at every step.

> **Key Insight:** Once a node is marked as *Visited*, its shortest path from the source is guaranteed in a graph with non-negative weights because any other path to this node would involve at least one more positive edge, making the total distance longer.

Termination Conditions

Dijkstra's can stop early if specified for a **Target Node**. If the target node is extracted from the Priority Queue (or identified as the minimum distance node in the unvisited set), the algorithm can terminate immediately, as no shorter path can possibly be found thereafter.

Manual Execution & Trace Table (Part 2)

Professional implementation requires tracking state across multiple iterations. Consider a graph where D is the source. The trace table maintains the **Priority Queue** (or distance array state) at every step.

> **Key Insight:** Once a node is marked as *Visited*, its shortest path from the source is guaranteed in a graph with non-negative weights because any other path to this node would involve at least one more positive edge, making the total distance longer.

Termination Conditions

Dijkstra's can stop early if specified for a **Target Node**. If the target node is extracted from the Priority Queue (or identified as the minimum distance node in the unvisited set), the algorithm can terminate immediately, as no shorter path can possibly be found thereafter.